

فاز ۲۱: میکروسرویس‌ها (Microservices) با

Spring Cloud

میکروسرویس‌ها یکی از مهم‌ترین مباحث برای بک‌اندکاران حرفه‌ای هستند. در مصاحبه‌های سطح بالا حتماً ازت می‌پرسن.

۲۱.۱ Microservices چیست؟

تعریف

میکروسرویس‌ها به سبک معماری هستند که در اون یک برنامه بزرگ به چندین سرویس کوچک و مستقل تقسیم میشه. هر سرویس:

- یک Business Capability رو پیاده‌سازی می‌کنه

- مستقل از بقیه Deploy میشه

- با HTTP یا Message Queue ارتباط برقرار می‌کنه

- می‌تونه با زبان‌های مختلف نوشته بشه

مقایسه Monolith vs Microservices

1		
2		
3	Monolith (تکنولوژی)	Microservices
4		
5	چندین سرویس کوچک	یک برنامه یکپارچه
6	JAR همه چیز توی یک	جداگانه JAR هر سرویس
7	هر سرویس دیتابیس خودش رو داره	یک دیتابیس مشترک
8	هر سرویس مستقل Deploy = تغییر	میشه Deploy کل برنامه
9	خطا فقط اون سرویس رو می‌گیره	خطا = کل برنامه کرش می‌کنه
10	مقیاس‌پذیری: فقط سرویس مورد نیاز	مقیاس‌پذیری: کل برنامه
11		
12		

مزایا و معایب معماری Microservices

مزایا	معایب
مقیاس‌پذیری بالا	پیچیدگی زیاد
جدا بودن تیم‌ها (استقلال تیم‌ها)	ارتباطات شبکه (Latency و مسائل شبکه)
Deploy مستقل (هر سرویس جداگانه)	مدیریت داده (تراکنش‌های توزیع شده)
خطا فقط یک سرویس را تحت تأثیر قرار می‌دهد	دیباگ سخت‌تر (ردیابی در چند سرویس)
انتخاب تکنولوژی آزاد برای هر سرویس	هزینه بیشتر (زیرساخت، عملیات)

نکات تکمیلی:

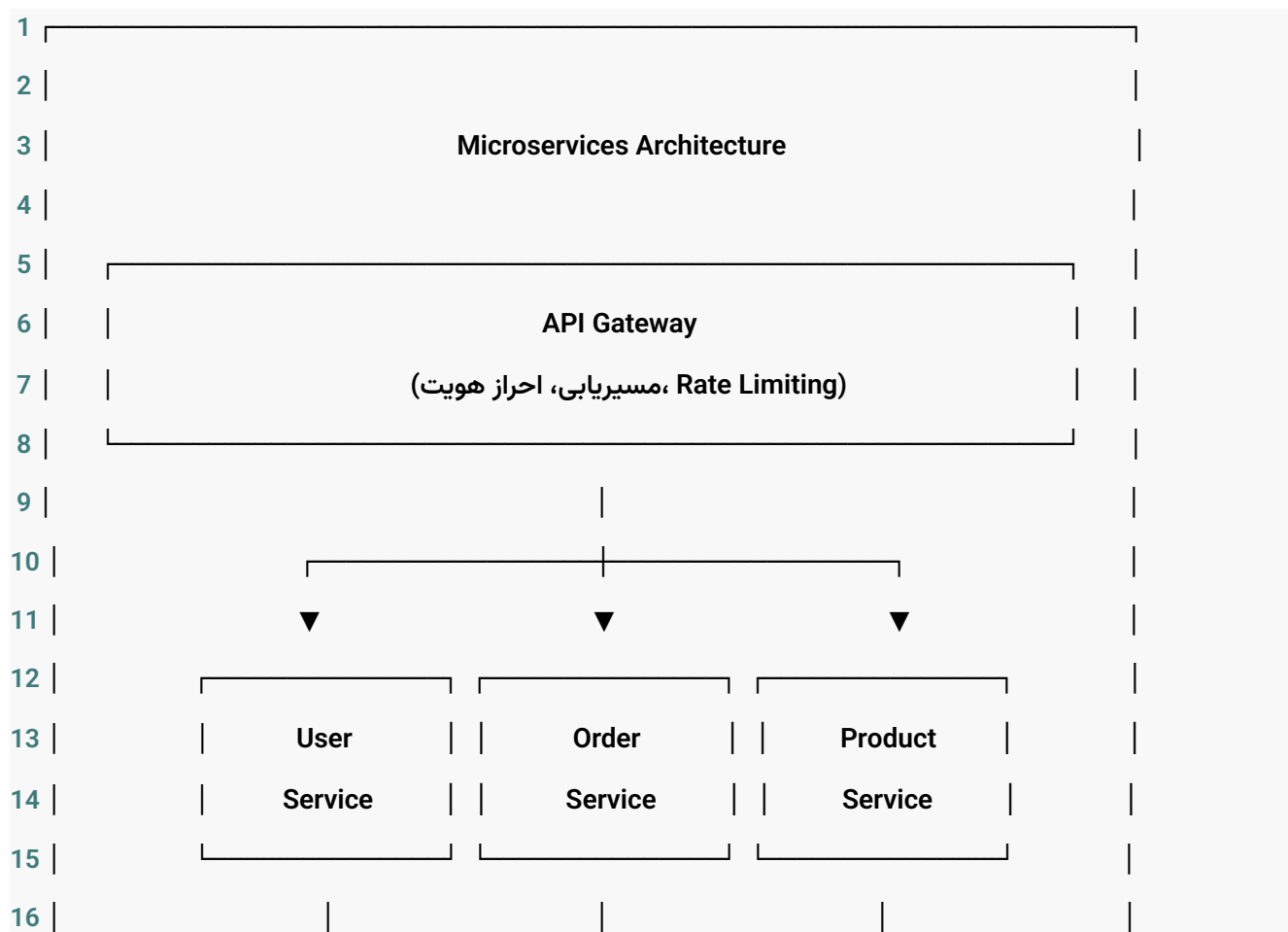
• مقیاس‌پذیری: هر سرویس به صورت مستقل مقیاس‌پذیر است و می‌توان فقط سرویس‌های پرتراфик را مقیاس کرد.

• استقلال تیم‌ها: هر تیم می‌تواند روی سرویس خود کار کند بدون وابستگی به تیم‌های دیگر.

• مدیریت داده: چالش اصلی در Microservices، مدیریت تراکنش‌ها و یکپارچگی داده بین سرویس‌هاست.

• هزینه بیشتر: نیاز به مدیریت چندین سرویس، لاگ‌گیری متمرکز، کشف سرویس و زیرساخت پیچیده‌تر دارد.

۲۱.۲ معماری میکروسرویس‌ها





Spring Cloud Netflix ۲۱.۳

وابستگی‌ها

```

1 <dependencyManagement>
2   <dependencies>
3     <dependency>
4       <groupId>org.springframework.cloud</groupId>
5       <artifactId>spring-cloud-dependencies</artifactId>
6       <version>2022.0.0</version>
7       <type>pom</type>
8       <scope>import</scope>
9     </dependency>
10  </dependencies>
11 </dependencyManagement>
12
13 <dependencies>
14   <!-- Eureka Server -->
15   <dependency>

```

```
16     <groupId>org.springframework.cloud</groupId>
17     <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
18 </dependency>
19
20 <!-- Eureka Client -->
21 <dependency>
22     <groupId>org.springframework.cloud</groupId>
23     <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
24 </dependency>
25
26 <!-- API Gateway -->
27 <dependency>
28     <groupId>org.springframework.cloud</groupId>
29     <artifactId>spring-cloud-starter-gateway</artifactId>
30 </dependency>
31
32 <!-- Circuit Breaker -->
33 <dependency>
34     <groupId>org.springframework.cloud</groupId>
35     <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
36 </dependency>
37
38 <!-- Load Balancer -->
39 <dependency>
40     <groupId>org.springframework.cloud</groupId>
41     <artifactId>spring-cloud-starter-loadbalancer</artifactId>
```

```
42 </dependency>
```

```
43 </dependencies>
```

Service Discovery (Eureka) ۲۱.۴

Eureka Server

```
1 @SpringBootApplication
2 @EnableEurekaServer
3 public class DiscoveryApplication {
4     public static void main(String[] args) {
5         SpringApplication.run(DiscoveryApplication.class, args);
6     }
7 }
8 # application.properties (Eureka Server)
9 server.port=8761
10 eureka.client.register-with-eureka=false
11 eureka.client.fetch-registry=false
12 eureka.server.wait-time-in-ms-when-sync-empty=0
```

Eureka Client

```
1 @SpringBootApplication
2 @EnableDiscoveryClient
3 public class UserServiceApplication {
4     public static void main(String[] args) {
5         SpringApplication.run(UserServiceApplication.class, args);
```

```

6  }
7  }
8  # application.properties (Eureka Client)
9  spring.application.name=user-service
10 server.port=8081
11
12 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka/
13 eureka.instance.prefer-ip-address=true
14 eureka.instance.instance-id=${spring.application.name}:${server.port}

```

استفاده از Service Discovery

```

1 @Service
2 @Slf4j
3 public class OrderService {
4
5     @Autowired
6     private LoadBalancerClient loadBalancer;
7
8     @Autowired
9     private RestTemplate restTemplate;
10
11     @Autowired
12     private DiscoveryClient discoveryClient;
13
14     // ۱. استفاده از LoadBalancerClient
15     public User getUserByUsername(String username) {

```

```

16     ServiceInstance instance = loadBalancer.choose("user-service");
17     String url = "http://" + instance.getHost() + ":" + instance.getPort() + "/api/users/" + username;
18     return restTemplate.getForObject(url, User.class);
19 }
20
21 // ۲. استفاده از ۲. DiscoveryClient
22 public List<User> getAllUsers() {
23     List<ServiceInstance> instances = discoveryClient.getInstances("user-service");
24     // ...
25 }
26
27 // ۳. با RestTemplate + @LoadBalanced
28 @Bean
29 @LoadBalanced
30 public RestTemplate restTemplate() {
31     return new RestTemplate();
32 }
33 // ۴. استفاده از ۴. RestTemplate (با LoadBalanced)
34 public User getUserByName(String username) {
35     return restTemplate.getForObject(
36         "http://user-service/api/users/" + username,
37         User.class
38     );
39 }
40 }

```


API Gateway ۲۱.۵

Gateway Service

```
1 @SpringBootApplication
2 @EnableDiscoveryClient
3 public class GatewayApplication {
4     public static void main(String[] args) {
5         SpringApplication.run(GatewayApplication.class, args);
6     }
7 }
8 # application.yml
9 server:
10  port: 8080
11
12 spring:
13  application:
14    name: api-gateway
15
16  cloud:
17    gateway:
18      routes:
19        - id: user-service
20          uri: lb://user-service
21          predicates:
22            - Path=/api/users/**
```

```
23
24   - id: order-service
25     uri: lb://order-service
26     predicates:
27       - Path=/api/orders/**
28
29   - id: product-service
30     uri: lb://product-service
31     predicates:
32       - Path=/api/products/**
33
34   # Global filters
35   default-filters:
36     - name: CircuitBreaker
37       args:
38         name: default
39         fallbackUri: forward:/fallback
40     - name: Retry
41       args:
42         retries: 3
43         statuses: SERVER_ERROR
44
45   management:
46     endpoints:
47       web:
48     exposure:
```

49 include: health,info,gateway

Gateway Filters

```

1 @Configuration
2 public class GatewayConfig {
3
4     // ۱. Custom Filter
5     @Bean
6     public GlobalFilter customFilter() {
7         return (exchange, chain) -> {
8             // route قبل از
9             ServerHttpRequest request = exchange.getRequest();
10            log.info("Request: {}", request.getMethod(), request.getURI());
11
12            // ادامه
13            return chain.filter(exchange).then(Mono.fromRunnable(() -> {
14                // route بعد از
15                ServerHttpResponse response = exchange.getResponse();
16                log.info("Response status: {}", response.getStatusCode());
17            }));
18        };
19    }
20
21    // ۲. Rate Limiter Filter
22    @Bean
23    public RedisRateLimiter redisRateLimiter() {

```

```

24     return new RedisRateLimiter(10, 20, 1); // 10 requests per second
25 }
26 }

```

Circuit Breaker (Resilience4j) ۲۱.۶

فعال سازی

```

1 @Configuration
2 @EnableCircuitBreaker
3 public class CircuitBreakerConfig { ... }

```

استفاده

```

1 @Service
2 @Slf4j
3 public class PaymentService {
4
5     @Autowired
6     private PaymentClient paymentClient;
7
8     // ۱. Circuit Breaker و Fallback
9     @CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackPayment")
10    public Payment processPayment(PaymentRequest request) {
11        log.info("Processing payment...");
12        return paymentClient.process(request);
13    }

```

```

14
15 // ٢. Fallback Method
16 public Payment fallbackPayment(PaymentRequest request, Throwable t) {
17     log.error("Payment fallback: {}", t.getMessage());
18     return Payment.builder()
19         .status("FAILED")
20         .message("Payment service unavailable")
21         .build();
22 }
23
24 // ٣. Retry
25 @Retry(name = "paymentService", fallbackMethod = "fallbackPayment")
26 public Payment processPaymentWithRetry(PaymentRequest request) {
27     return paymentClient.process(request);
28 }
29
30 // ٤. Rate Limiter
31 @RateLimiter(name = "paymentService")
32 public Payment processPaymentWithRateLimit(PaymentRequest request) {
33     return paymentClient.process(request);
34 }
35
36 // ٥. Bulkhead (محدودیت همزمان)
37 @Bulkhead(name = "paymentService", type = Bulkhead.Type.THREADPOOL)
38 public Payment processPaymentWithBulkhead(PaymentRequest request) {
39     return paymentClient.process(request);

```

40 }

41 }

پیکربندی Resilience4j

```
1 resilience4j:
2   circuitbreaker:
3     instances:
4       paymentService:
5         sliding-window-size: 10
6         failure-rate-threshold: 50
7         wait-duration-in-open-state: 10s
8         minimum-number-of-calls: 5
9
10    retry:
11      instances:
12        paymentService:
13          max-attempts: 3
14          wait-duration: 2s
15
16    ratelimiter:
17      instances:
18        paymentService:
19          limit-for-period: 10
20          limit-refresh-period: 1s
21
22    bulkhead:
```

```
23 instances:
24     paymentService:
25         max-concurrent-calls: 5
```

Distributed Tracing (Sleuth + Zipkin) ۲۱.۷

```
1 <dependency>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-starter-sleuth</artifactId>
4 </dependency>
5 <dependency>
6   <groupId>org.springframework.cloud</groupId>
7   <artifactId>spring-cloud-starter-zipkin</artifactId>
8 </dependency>
9 spring.sleuth.sampler.probability=1.0
10 spring.zipkin.base-url=http://localhost:9411
```

استفاده از Trace ID

```
1 @Service
2 @Slf4j
3 public class OrderService {
4
5     @Autowired
6     private Tracer tracer;
7
8     public void processOrder(Order order) {
```

```

9    // Trace ID خودکار در لاگ‌ها ظاهر میشه
10   log.info("Processing order: {}", order.getId());
11
12   Span span = tracer.nextSpan().name("processOrder");
13   try (Tracer.SpanInScope ws = tracer.withSpan(span.start())) {
14       // پردازش
15       log.info("Order processed");
16   } finally {
17       span.end();
18   }
19 }
20 }

```

۲۱.۸ ! سوالات مصاحبه‌ای

? سوال ۱: میکروسرویس چیه و چرا استفاده میشه؟

جواب: Microservices به معماری هست که برنامه رو به چندین سرویس کوچک و مستقل تقسیم می‌کنه. مزایا: مقیاس‌پذیری بالا، Deploy مستقل، انتخاب تکنولوژی آزاد، خطا فقط یک سرویس.

? سوال ۲: Eureka چیه؟

جواب: Eureka به سرویس ثبت و کشف (Service Discovery) هست. سرویس‌ها خودشون رو در Eureka ثبت می‌کنن و سرویس‌های دیگه می‌تونن همونجا پیدا بشن.

? سوال ۳: API Gateway چیه و چرا استفاده میشه؟

جواب: API Gateway به نقطه ورودی واحد برای همه سرویس‌هاست. کارها: مسیریابی درخواست‌ها، احراز هویت، Rate Limiting، Caching، Logging.

? سوال ۴: Circuit Breaker چیه؟

جواب: Circuit Breaker به الگو برای مدیریت خطا در سرویس‌های دیگه هست. وقتی یه سرویس خطا بده، Circuit Breaker باز میشه و درخواست‌ها دیگه به اون سرویس نمی‌رن (تا بازیابی).

? سوال ۵: فرق Monolith و Microservices چیه؟

جواب: Monolith یک برنامه یکپارچه است، Microservices چندین سرویس کوچک. Monolith ساده‌تر ولی کمتر مقیاس‌پذیر، Microservices پیچیده‌تر ولی مقیاس‌پذیرتر.